

Práctica Semana 9

Curso: Programación Orientada a Objetos II

Tema: Manejo avanzado de excepciones, logging y monitoreo en tiempo real

Duración: 2 horas

OBJETIVO

- Implementar jerarquías de excepciones personalizadas en C++.
- Integrar un sistema de logging para registrar eventos críticos y de depuración.
- Simular un monitoreo en tiempo real con registro de errores y métricas del sistema.

MARCO TEÓRICO

1. **Excepciones personalizadas:** permiten representar errores específicos con mayor semántica que `std::exception`.
2. **Logging:** es el registro estructurado de eventos que ayuda a la depuración, análisis de errores y auditoría. En C++ se puede implementar con librerías como `spdlog` o de forma manual con archivos de texto.
3. **Monitoreo en tiempo real:** implica capturar métricas y estados de ejecución durante la operación del sistema.

CÓDIGO DE LA PRÁCTICA

Ejemplo en C++:

```
#include <iostream>
#include <fstream>
#include <stdexcept>
#include <ctime>

// Excepción personalizada
class DivisionByZeroException : public std::runtime_error {
public:
    DivisionByZeroException()
        : std::runtime_error("Error: División entre cero detectada.")
    {}
};

// Logger simple
class Logger {
private:
    std::ofstream logfile;
public:
    Logger(const std::string& filename) {
        logfile.open(filename, std::ios::app);
    }
    ~Logger() {
        if (logfile.is_open()) logfile.close();
    }
    void log(const std::string& message) {
        time_t now = time(0);
        logfile << "[" << ctime(&now) << "]" << message << std::endl;
    }
};

// Función que genera una excepción
double dividir(double a, double b) {
    if (b == 0) throw DivisionByZeroException();
    return a / b;
}
```

```

}

int main() {
    Logger logger("system.log");

    try {
        std::cout << "Intentando dividir 10 / 0 ..." << std::endl;
        double resultado = dividir(10, 0);
        std::cout << "Resultado: " << resultado << std::endl;
    }
    catch (const DivisionByZeroException& ex) {
        std::cerr << ex.what() << std::endl;
        logger.log(ex.what()); // Se registra el error en el log
    }
    catch (const std::exception& ex) {
        std::cerr << "Excepción genérica: " << ex.what() << std::endl;
        logger.log("Excepción genérica: " + std::string(ex.what()));
    }

    logger.log("Ejecución finalizada correctamente.");
    std::cout << "Verifica el archivo system.log para los registros."
    << std::endl;

    return 0;
}

```

DESARROLLO

1. Compila y ejecuta el programa.
2. Revisa que el archivo **system.log** se haya creado con los errores registrados.
3. Simula distintos errores (división entre cero, valores negativos, entradas no numéricas).
4. Implementa un ciclo que divida una lista de números y registre los errores de forma continua (monitoreo en tiempo real).

TRABAJO DE INVESTIGACIÓN

1. Investiga y documenta **3 librerías profesionales de logging en C++** (ejemplo: spdlog, Boost.Log, log4cpp).
2. Explica cómo se integran con **herramientas de monitoreo en tiempo real** (Ejemplo: Grafana, Kibana, Prometheus).
3. Adjunta un **informe en Word** con comparaciones y ejemplos.